Lista de Exercícios — Sua primeira rede em PyTorch

Prof. Me. Gabriel Duarte

Disciplina: Inteligência Artificial

Objetivo

Você já entendeu como uma MLP funciona por dentro (forward e retropropagação na mão). Agora o objetivo é **outro**: usar o **PyTorch** para **resolver um problema de verdade** em poucas linhas. Você vai **definir a rede**, **treinar** e **avaliar** um classificador para a base **Iris** (3 espécies de flor). As derivadas **não** são mais problema seu — o PyTorch calcula com `loss.backward()` — então o foco passa a ser **montar o pipeline** e **interpretar os resultados**. Roda no **Google Colab** (CPU basta, ∼2 segundos de treino).

1 O problema e os dados

A base **Iris** tem **150 amostras**, **4 características** por amostra (comprimento e largura da sépala e da pétala) e **3 classes** (*setosa*, *versicolor*, *virginica*). É um problema de **classificação multiclasse**.

A parte chata (carregar, dividir, padronizar) já vem pronta — **não mexa nela**, o exercício é o PyTorch:

```
import numpy as np, torch, torch.nn as nn
from sklearn.datasets import load_iris

SEED = 42
torch.manual_seed(SEED)
iris = load_iris()
X = torch.tensor(iris.data, dtype=torch.float32)      # (150, 4)
y = torch.tensor(iris.target, dtype=torch.long)   # (150,) 3 classes
# divisão treino/teste (80/20) reprodutível
perm = torch.randperm(len(X),
generator=torch.Generator().manual_seed(SEED))
n_tr = int(0.8 * len(X))
Xtr, ytr = X[perm[:n_tr]], y[perm[:n_tr]]
Xte, yte = X[perm[n_tr:]], y[perm[n_tr:]]
# padronização com as estatísticas SÓ do treino
mu, sd = Xtr.mean(0), Xtr.std(0)
Xtr, Xte = (Xtr - mu) / sd, (Xte - mu) / sd
```

Resultado esperado dessa primeira parte: **120 amostras de treino** e **30 de teste**.

Duas ideias antes de codar

(a) Uma camada `nn.Linear(entrada, saida)` faz exatamente $\mathbf{Wx} + \mathbf{b}$ — é o mesmo `X @ W + b` da aula, só empacotado. (b) A `nn.CrossEntropyLoss` **já** aplica o `softmax` internamente, então a sua rede deve devolver **logits** (saída crua, sem ativação no final), **não** probabilidades.

2 Exercício 1 — Defina a rede

1. Crie uma classe `MLP(nn.Module)` com a arquitetura **4** → **16** → **16** → **3**: duas camadas ocultas de 16 neurônios com **ReLU** e uma camada de saída de 3 neurônios. A forma mais curta é

juntar tudo num `nn.Sequential` dentro do `__init__`.

2. Escreva o método `forward(self, x)` devolvendo a saída da rede. **Sem softmax no final** (veja a caixa laranja acima).

3. Instancie o modelo e **imprima** o número de parâmetros treináveis: `sum(p.numel() for p in modelo.parameters())`. **Esperado: 403** (some as contas: $4{\cdot}16+16$, $16{\cdot}16+16$ e $16{\cdot}3+3$). Se der outro valor, a arquitetura está diferente da pedida.

3 Exercício 2 — Treine

O laço de treino em PyTorch tem **cinco passos, sempre na mesma ordem**:

```
otimizador.zero_grad() → saida = modelo(Xtr) → perda = criterio(saida, ytr) →
                        perda.backward() → otimizador.step()
```

1. Crie a função de perda e o otimizador: `criterio = nn.CrossEntropyLoss()` e `otimizador = torch.optim.Adam(modelo.parameters(), lr=1e-2)`.

2. Escreva o laço de **200 épocas** com os cinco passos acima, usando **o lote inteiro** (todas as 120 amostras de uma vez — não precisa de `DataLoader` aqui). Guarde a perda de cada época numa lista `hist`.

3. **Imprima** a perda nas épocas $0, 1, 2, 5, 10, 25, 50, 100, 199$. Confira: ela começa em $\approx$ **1,10** (que é $\ln 3 \approx 1{,}0986$ — o valor exato de um modelo que ainda chuta igualmente entre as 3 classes) e termina abaixo de 0,001.

4. Plote `hist` $\times$ época. A curva deve cair rápido no início e achatar no fim.

5. **Responda** (2–3 frases): o que aconteceria se você **não** chamasse `zero_grad()` a cada época? Por que o `backward()` precisa vir *depois* do cálculo da perda?

O erro mais comum da lista

Se a perda não cair (fica perto de 1,1 para sempre), quase sempre é `zero_grad()` esquecido, `backward()` esquecido, ou `otimizador.step()` esquecido. Confira a ordem dos cinco passos antes de suspeitar do modelo.

4 Exercício 3 — Avalie

Para avaliar, a rede deve entrar em **modo de avaliação** e o cálculo *não* pode construir grafo de gradiente:

```
modelo.eval()
with torch.no_grad():
pred = modelo(Xte).argmax(dim=1)
```

1. Calcule a **acurácia** no treino e no teste: `(pred == y).float().mean().item()`.

2. Monte a **matriz de confusão** do teste (linha = classe verdadeira, coluna = classe prevista). Você pode fazer com dois `for` e um `np.zeros((3,3))`.

3. **Analise**: quais classes o modelo confunde? Ele erra *setosa*? Olhe a matriz e explique em 3–5 frases por que o erro se concentra no par que se concentra.

4. **Confira a coerência do `softmax`**: aplique `torch.softmax(logits, dim=1)` nas 5 primeiras amostras de teste e verifique que **cada linha soma 1**. Aproveite para conferir que `nn.CrossEntropyLoss()` sobre os logits dá o mesmo resultado que `nn.NLLLoss()` sobre `log_softmax(logits)`.

5 Desafios (escolha pelo menos um)

1. **Trocar o otimizador.** Rode o mesmo treino (200 épocas, mesma semente) com `SGD(lr=1e-2)`, com `SGD(lr=0.1, momentum=0.9)` e com `Adam(lr=1e-2)`. Monte uma tabela com perda final e acurácia de teste. *Pergunta:* por que SGD com `lr=1e-2` fica tão atrás?

2. **Tamanho da camada oculta.** Rode com `n_oculta` $= 2, 4, 16, 64$ e monte uma tabela com acurácia de teste e número de parâmetros. *Pergunta:* mais parâmetros significa melhor resultado aqui? (Olhe o equilíbrio entre acurácia de **treino** e de **teste**.)

3. **Número de épocas.** Rode com $20, 50, 100$ e 200 épocas. *Pergunta:* treinar mais tempo sempre melhora o resultado no *teste*? Relacione com o que você viu na acurácia de treino.

4. **(Extra)** Troque a divisão por um `DataLoader` com `batch_size=16` e `shuffle=True` e observe o que muda na curva de perda.

6 Entrega

Entregue um **notebook Jupyter** (`.ipynb`) executado no Colab, comentado em PT-BR, com a curva de perda, a matriz de confusão e as respostas escritas. Vale 10,0 pontos:

Critério	Valor	Como é avaliado
Exercício 1 (a rede)	2,0	Arquitetura correta; 403 parâmetros; logits sem softmax
Exercício 2 (o treino)	3,5	Os cinco passos na ordem; perda caindo até $< 0{,}001$; curva plotada
Exercício 3 (avaliação)	3,0	Acurácias corretas; matriz de confusão; análise do par confundido; coerência do softmax
Um desafio	1,0	Tabela comparativa + resposta à pergunta
Clareza e comentários	0,5	Código comentado em PT-BR, reprodutível no Colab

Sinais de que está tudo certo (autochecagem)

Perda inicial $\approx 1{,}10$; perda final $< 0{,}001$; acurácia de **treino** $= 1{,}00$ (120/120); acurácia de **teste** $\approx 0{,}90$ (27/30); a rede **não** erra nenhuma *setosa*. Se a acurácia de teste der exatamente 1,00, desconfie: pode haver **vazamento** (você padronizou com as estatísticas do conjunto todo?) ou você mediu no treino sem querer. Os valores exatos e o código de referência estão no **notebook resolvido** do professor.